

KOHLER AI Bathroom Designer & Planner

Phase 2 Case Study Challenge: System Design & AI Documentation

1. PROJECT OVERVIEW

The KOHLER AI Bathroom Designer is an interactive 3D design assistant engineered to generate optimized product combinations based on dimensional and functional constraints. The system accepts core parameters—**bathroom dimensions, budget, aesthetic preferences, and functional needs**—to compute a tailored environment.

The solution actively curates from four primary product categories:

- Toilets (including Smart & Standard configurations)
- Showers and Bathtubs
- Vanities (Cabinets, Counters, Basins)
- Faucets and Hardware

Generated product bundles are immediately visualized inside a responsive, interactive 3D bathroom environment. Following the initial generation, users can refine the spatial and aesthetic design using natural-language instructions.

2. AI / INTELLIGENT SYSTEM APPROACH

The current prototype utilizes a deterministic, robust architecture designed for speed and explainability. **The prototype does NOT call an external LLM (e.g., ChatGPT, Gemini) or AI API.**

Instead, intelligent processing is divided into two transparent systems:

1. **Natural-Language Interaction:** Implemented via a transparent rule-based parser that maps linguistic intents to structural configuration variables.
2. **Recommendation Engine:** Utilizes an exhaustive combinatorial search algorithm over the product catalog to evaluate, score, and select the mathematically optimal product bundle.

This localized architecture was deliberately chosen for the following advantages:

- **Deterministic:** Identical constraints always yield consistent results.
- **Fast:** Zero network latency; runs entirely client-side.
- **Explainable:** Scoring parameters are fully transparent.
- **No API Key:** Removes external dependencies and operational costs.
- **Debuggable:** Clear separation between language parsing and logic.
- **Extensible:** Easily upgradeable to an LLM semantic layer later.

3. NATURAL-LANGUAGE PROMPT DOCUMENTATION

The embedded parser relies on semantic matching to update the underlying parameters. Below is a formalized list of supported intents and their exact functional interpretations within the engine.

Category	User Prompt (Example)	Expected Interpretation
Luxury	"Make it more luxurious"	Increase the luxury / quality weighting used by the optimizer.
Value	"Make it cheaper"	Move the optimization toward more economical product choices.
Extreme Value	"Cheapest possible"	Strongly prioritize economical choices above aesthetic scoring.
Extreme Luxury	"Money is no object"	Maximize the luxury / quality preference to its highest limit.
Theme Change	"Go industrial"	Switch the selected environment theme to <i>Industrial Loft</i> .
Zen Theme	"Make it feel more zen"	Switch the selected environment theme to <i>Japanese Zen</i> .
Bathtub	"Add a tub"	Toggle boolean to allow bathtub products during combinatorial optimization.
Shower Only	"Shower only"	Restrict the recommendation exclusively to shower configurations.
Relative Budget	"Increase my budget by 20%"	Calculate and apply a +20% increase to the current numerical budget.
Budget Reduction	"Cut the cost by 15%"	Calculate and apply a -15% reduction to the current numerical budget.
Budget Cap	"Don't cross ₹2L"	Extract numerical entity and set the maximum budget to ₹200,000.
Power Constraint	"No power"	Exclude all power-dependent products (e.g., smart toilets) from results.

4. MULTI-INTENT PROMPTS

The system is capable of extracting and applying multiple parametric transformations from a single natural-language sentence. The language layer never directly selects individual SKUs; it only alters the constraint matrix sent to the primary optimization engine.

5. PARAMETER MAPPING

The parser acts as a translation matrix, converting detected textual intents into explicitly typed state variables managed by the central configuration store.

Natural-language intent	Internal parameter	Effect
More luxurious	luxuryWeight	Increases quality/luxury preference
Cheaper	luxuryWeight	Moves toward value-oriented results
Industrial	theme	Selects Industrial Loft
Zen	theme	Selects Japanese Zen
Add a tub	wantTub	Allows bathtub options
Shower only	wantTub	Removes bathtub options
No power	hasPower	Excludes power-dependent products
Increase budget	budget	Raises budget
Reduce budget	budget	Lowers budget
Budget cap	budget	Sets maximum budget

6. RECOMMENDATION ENGINE

Independent from the language parser, the Recommendation Engine performs an **exhaustive combinatorial search** across the active product catalog. The current prototype maintains a highly curated matrix of **30 distinct products** across four categories (Toilet, Shower/Bath, Vanity, Faucet).

Each product is strictly modeled with quantifiable attributes:

- Price / Cost
- Physical Dimensions (W, H, D)
- Quality / Luxury Index
- Theme Compatibility Matrices
- Functional Compatibility (e.g., Power needs)

The engine generates possible topological combinations and scores them. Budget and space act as soft constraints (penalties)—ensuring the system can still gracefully degrade to the "best available" fallback solution if a perfect dimensional/financial fit is mathematically impossible.

$$\begin{aligned} \text{Final Score} = & (\text{Quality Weight}) + (\text{Theme Fit}) + (\text{Value Fit}) \\ & - (\text{Budget Penalty Factor}) \\ & - (\text{Space Constraint Penalty}) \end{aligned}$$

7. THREE OPTIMIZATION PROFILES

The combinatorial solver does not generate merely one bundle; it dynamically shifts its algorithmic weights to generate three distinct operational profiles simultaneously. These are not merely re-labeled recommendations, but fundamentally different weighting strategies over the same catalog.

VALUE Applies a heavy multiplier to cost efficiency, maintaining a strong preference toward economical product choices while meeting base constraints.	BALANCED The default standard. Achieves an equilibrium between product quality, theme compatibility, and overall cost footprint.	LUXURY De-emphasizes the strict budget penalty slightly to place greater operational emphasis on quality, materials, and premium choices.
--	--	---

8. 3D VISUALIZATION ARCHITECTURE

The output matrix of the recommendation engine is bound directly to a WebGL rendering context. This provides immediate, interactive visual feedback representing the algorithmic choices.

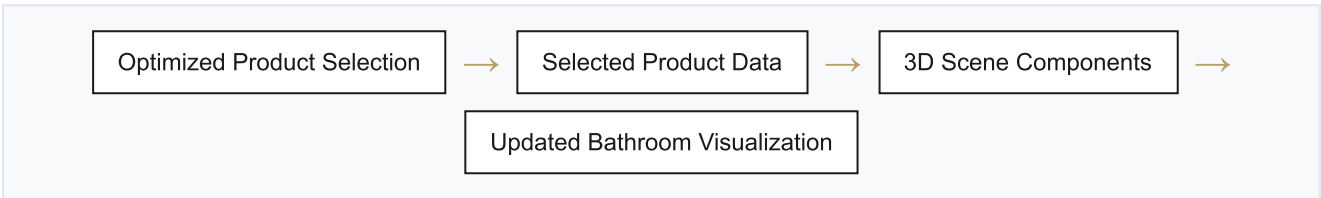
Core Rendering Technology:

- **Three.js:** Underlying WebGL 3D engine.
- **React Three Fiber (R3F):** React reconciler bridging state variables to 3D meshes.
- **@react-three/drei:** Helper library for controls and environment staging.

The real-time scene constructs graphical representations of the selected Toilet, Vanity, Faucet, and Shower/Bathtub, situated within an architecturally bounded room containing appropriate mirrors and lighting models.

Implemented Interaction Features:

- Full Orbit Controls (Pan, Zoom, Rotate)
- Wide architectural camera view
- Focused Vanity camera view
- Focused Shower camera view
- Theme-dependent room appearance (textures/colors swap based on engine state)
- Snapshot rendering & export



9. END-TO-END EXECUTION WALKTHROUGH

The following traces a complete user interaction loop demonstrating how dimensional constraints, optimization, and natural-language refinement interface seamlessly.

INITIAL SCENARIO

A user desires a premium aesthetic but is constrained by a strict renovation budget.

Step 1: Constraint Ingestion

User inputs base parameters: Room dimensions (8x8ft), Budget (₹180,000), Japanese Zen style, Power available (True), Tub preference (True).

Step 2: Initial Optimization Generation

The engine evaluates the catalog and returns three distinct profiles mapped to the spatial constraints:

- *Value* (₹110,000 - basic tub, standard vanity)
- *Balanced* (₹155,000 - upgraded fixtures)
- *Luxury* (₹175,000 - premium materials)

STEP 3: USER NATURAL-LANGUAGE REFINEMENT

"Make it more luxurious but don't cross ₹2.5L"

Step 4: Parameter Extraction & Reprocessing

A. Natural-language input parsing:

Identifies intent for higher quality tier and extracts explicit currency cap.

B. Parameter adjustment:

Sets `luxuryWeight += 2.0`; Updates budget to ₹250,000.

C. Optimizer re-run:

Algorithm recalculates scores allowing for previously excluded high-tier SKUs.

D. Updated Recommendation:

New bundle generated containing smart toilet and freestanding tub at ₹235,000.

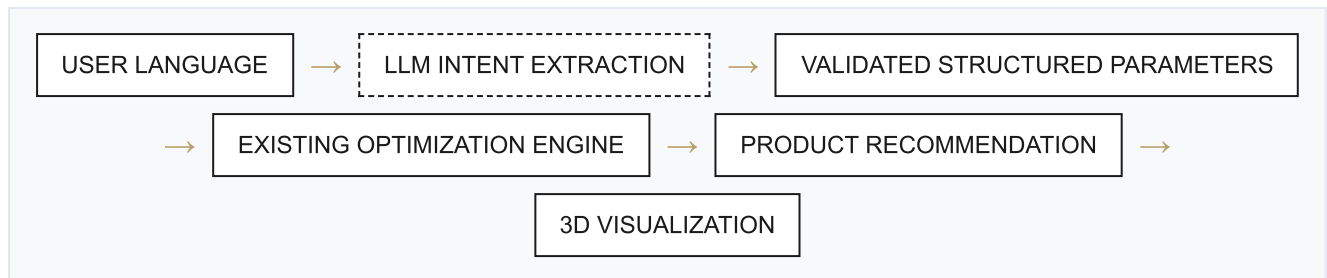
E. Updated 3D Scene:

React Three Fiber mounts new geometric assets into the viewport instantly.

10. FUTURE LLM INTEGRATION

While the current implementation strictly leverages a deterministic rule-based parser for latency and stability, the architecture is specifically decoupled to allow a seamless future integration of a Large Language Model (LLM).

In a production deployment, the future LLM would **NOT** be responsible for directly selecting products or calculating dimensions. Instead, it would serve purely as an advanced semantic extraction layer:



Validation logic sits between the LLM and the optimizer to ensure hallucinations do not crash the engine. Below is a sample system prompt for this future upgrade:

```
"You are a bathroom-design intent parser. Convert the user's natural-language request into structured parameters including budget, theme, luxuryWeight, wantTub and hasPower. Preserve parameters that the user did not request to change. Only return valid JSON values supported by the application."
```

11. IMPLEMENTATION FILES

The functionality described in this document maps directly to the following core source files in the project repository:

- `src/engine/nlEdit.js`
Handles natural-language parsing, regex matching, and parameter extraction.
- `src/engine/optimize.js`
Contains the exhaustive combinatorial recommendation and scoring algorithm.
- `src/data/catalog.js`
Defines the structured product database and aesthetic theme compatibility matrices.
- `src/store/useConfigStore.js`
Zustand store managing application configuration, execution state, and history.
- `src/ui/RefineBar.jsx`
The interactive React component capturing user textual input for refinement.

12. PROTOTYPE LIMITATIONS

As a Phase 2 Case Study Challenge submission, this prototype establishes a foundation but acknowledges defined boundaries that represent opportunities for scale in production environments:

- Natural-language understanding is rule-based and regex-driven, lacking the zero-shot general comprehension of unrestricted conversational AI.
- The product catalog operates on a static, local dataset of 30 mockups rather than a live GraphQL/API connection to the actual KOHLER global product database.
- 3D fixture placement relies on intelligent heuristic positioning grids rather than a complete architectural collision-detection physics system.
- The optimizer uses an exact search, which is fast for 30 products but would require heuristic or ML-based pruning to run in real-time over 10,000+ SKUs.
- Image recognition and floor-plan upload understanding are excluded from this phase.

13. FUTURE ROADMAP

The architecture allows for progressive enhancement across data, logic, and visualization tiers.

CURRENT PROTOTYPE



REAL KOHLER PRODUCT DATA (SKUs, live pricing, availability)



ADVANCED SPATIAL REASONING (Plumbing layout, true collision rules)



IMAGE / FLOOR-PLAN UNDERSTANDING (Vision models for auto-dimensioning)



LLM-POWERED DESIGN ASSISTANT (Conversational context retention)



PHOTOREALISTIC / AR VISUALIZATION (WebXR integration)

KOHLER AI Bathroom Designer & Planner

"Turning bathroom requirements into personalized, constraint-aware and visual product solutions."